

ΕΓΓΡΑΦΟ ΠΕΡΙΓΡΑΦΗΣ ΠΡΩΤΟΚΟΛΛΟΥ

1. Περιγραφή Αρχιτεκτονικής (Client-Server)

Το προτεινόμενο σύστημα ανταλλαγής μηνυμάτων σε πραγματικό χρόνο βασίζεται στην αρχιτεκτονική **Client-Server** και χρησιμοποιεί το πρωτόκολλο μεταφοράς **UDP**. Καθώς το UDP είναι εκ φύσεως connectionless και αναξίοπιστο, η διαχείριση της κατάστασης των χρηστών (session state), η ανίχνευση αποσυνδέσεων και η επιβεβαίωση ταυτότητας υλοποιούνται εξολοκλήρου στο επίπεδο εφαρμογής (Application Layer).

1.1 Ο ρόλος του Server

Ο κεντρικός Server λειτουργεί ως ο μοναδικός διαιτητής, δρομολογητής μηνυμάτων και statekeeper της εφαρμογής. Τα βασικά δομικά του χαρακτηριστικά περιλαμβάνουν:

- Multi-threading:** Ο Server διαθέτει ένα κύριο thread που εκτελεί έναν βρόχο υποδοχής εισερχόμενων datagrams (μέσω της εντολής `recvfrom`). Παράλληλα, εκκινεί ένα ανεξάρτητο background thread (Timeout Daemon) το οποίο εκτελεί περιοδικούς ελέγχους για την εκκαθάριση ανενεργών χρηστών.
- Thread Safety:** Για την αποφυγή race conditions κατά την ταυτόχρονη προσπέλαση και τροποποίηση των κοινόχρηστων δομών (όπως τα λεξικά `clients`, `user_rooms` και `last_seen`), χρησιμοποιείται ένας μηχανισμός αμοιβαίου αποκλεισμού `threading.Lock()`.
- Persistence Layer (SQLite3):** Ο Server ενσωματώνει απευθείας τη βάση δεδομένων `chat_history.db`. Κάθε έγκυρο μήνυμα (Global, Room, Private) καταγράφεται άμεσα με χρονική σήμανση. Κατά την ανάκτηση του ιστορικού, ο Server εφαρμόζει κανόνες εξουσιοδότησης (Authorization Filters), διασφαλίζοντας ότι ένας χρήστης μπορεί να ανακτήσει μόνο τα ιδιωτικά μηνύματα που αφορούν τον ίδιο.

1.2 Ο ρόλος του Client

Η εφαρμογή παρέχει δύο εναλλακτικές μορφές Client: έναν CLI Client (γραμμή εντολών) και έναν full-featured GUI Client (βασισμένο στη βιβλιοθήκη `tkinter`). Και οι δύο εκδόσεις ακολουθούν την ίδια δικτυακή λογική:

- Ασύγχρονη Λήψη (Multi-threaded Receiving):** Για να μην παγώνει η διεπαφή του χρήστη κατά την αναμονή δεδομένων, ο Client εκκινεί ένα ξεχωριστό thread λήψης (`receive_messages`), το οποίο ακούει συνεχώς στο socket και ενημερώνει την οθόνη μόλις καταφθάσει ένα JSON πακέτο.
- Μηχανισμός Keep-Alive:** Ένα background thread αναλαμβάνει την υποχρέωση αποστολής περιοδικών πακέτων τύπου `PING` για τη διατήρηση της σύνδεσης.

2. Ορισμός Μορφής Μηνυμάτων και Τύπων

Η ανταλλαγή δεδομένων πραγματοποιείται αποκλειστικά με πακέτα μορφοποιημένα κατά το πρότυπο **JSON**, τα οποία κωδικοποιούνται σε bytes με τη μορφή **utf-8**.

2.1 Δομή του JSON Πακέτου (Format)

Κάθε πακέτο που διακινείται στο δίκτυο περιέχει μια αυστηρή δομή με τα ακόλουθα πεδία:

```
{
  "msg_id": 105,
  "action": "BROADCAST",
  "sender": "Aliko",
  "target": null,
  "payload": "Hello world!",
  "status": null
}
```

- msg_id** (Integer): Μοναδικός αύξων αριθμός μηνύματος που παράγεται τοπικά από τον αποστολέα για σκοπούς ιχνηλάτησης.
- action** (String): Ο τύπος της ενέργειας ή του αιτήματος.
- sender** (String): Το username του χρήστη (ή **"SERVER"** για συστημικά μηνύματα).
- target** (String / Null): Ο αποδέκτης-στόχος (όνομα δωματίου, username άλλου χρήστη ή τύπος ιστορικού).
- payload** (String / List / Null): Το κυρίως περιεχόμενο (κείμενο, σφάλμα ή πίνακας δεδομένων).
- status** (String / Null): Χρησιμοποιείται από τον Server στις απαντήσεις του (**"SUCCESS"** ή **"ERROR"**).

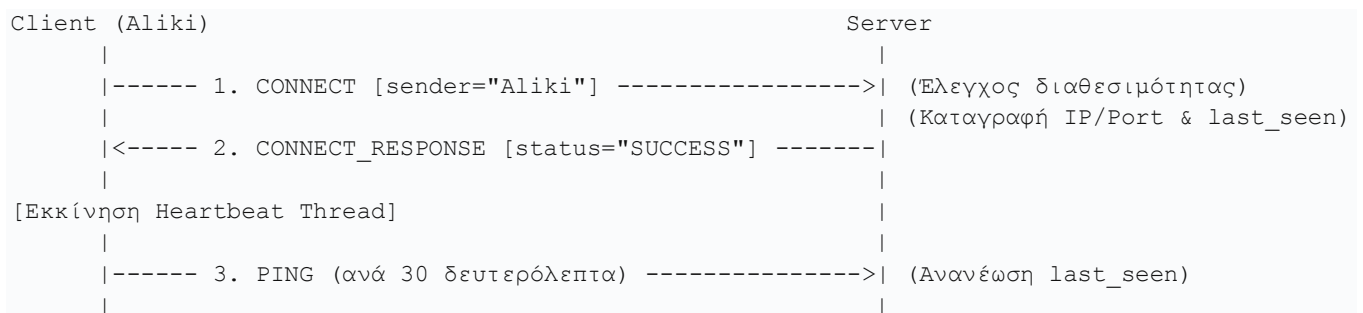
2.2 Υποστηριζόμενοι Τύποι Ενεργειών (Actions)

Action	Κατεύθυνση	Περιγραφή & Χρήση Πεδίων
CONNECT	Client → Server	Αίτημα αρχικής εγγραφής. Το πεδίο sender περιέχει το επιθυμητό username.
CONNECT_RESPONSE	Server → Client	Απάντηση έγκρισης ή απόρριψης. Επιστρέφει status="SUCCESS" ή "ERROR" .
DISCONNECT	Client → Server	Εθελούσια ειδοποίηση αποσύνδεσης για άμεση εκκαθάριση από τις δομές.
PING	Client → Server	Σήμα Heartbeat (Keep-Alive) που αποστέλλεται αυτόματα ανά 30 δευτερόλεπτα.

BROADCAST	Αμφίδρομο	Μήνυμα καθολικής εκπομπής. Ο Server το προωθεί σε όλους τους online χρήστες.
PRIVATE	Αμφίδρομο	Προσωπικό μήνυμα. Το πεδίο target περιέχει το username του παραλήπτη.
JOIN_ROOM	Client → Server	Αίτημα εισόδου σε δωμάτιο. Το target ορίζει το όνομα του δωματίου.
ROOM_MSG	Αμφίδρομο	Μήνυμα εντός δωματίου. Το target περιέχει το όνομα του τρέχοντος δωματίου.
ROOM_ANNOUNCEMENT	Server → Client	Συστημική ειδοποίηση προς τα μέλη ενός δωματίου (π.χ. είσοδος/έξοδος χρήστη).
LIST_USERS	Client → Server	Αίτημα λήψης της λίστας των ενεργών χρηστών.
LIST_USERS_RESPONSE	Server → Client	Επιστροφή της λίστας. Το payload περιέχει έναν πίνακα (Array) με τα usernames.
HISTORY_REQUEST	Client → Server	Αίτημα ιστορικού. target : "BROADCAST", "ROOM", ή "PRIVATE".
HISTORY_RESPONSE	Server → Client	Επιστροφή ιστορικού συνομιλιών ως πίνακας JSON αντικειμένων από την SQLite3.

3. Διαγράμματα Ροής (Sequence Diagrams)

3.1 Διαδικασία Εγγραφής & Σύνδεσης (CONNECT)



Διάγραμμα 1: Διαδικασία αυθεντικοποίησης και keep-alive.

3.2 Ανταλλαγή Μηνυμάτων σε Δωμάτιο (JOIN & ROOM_MSG)

```
Client (Aliki)                                Server                                Client (Aggelina)
|                                                |
|-- 1. JOIN_ROOM [target="diktia"] -->|        |
|                                        |-- 2. ROOM_ANNOUNCEMENT ----->|
|                                        (Payload: "Aliki joined...") |
|<-- 3. RESPONSE [status="SUCCESS"] --|        |
|                                        |
|-- 4. ROOM_MSG [target="diktia"] --->| (Αποθήκευση στην SQLite3) |
|    (Payload: "Γεια σας!")           |
|                                        |-- 5. ROOM_MSG [target="diktia"] -->|
|    (sender="Aliki", "Γεια σας!") |
```

Διάγραμμα 2: Συμμετοχή σε δωμάτιο και προώθηση μηνυμάτων.

3.3 Αίτημα και Ασφαλής Ανάκτηση Ιστορικού (HISTORY)

```
Client (Aliki)                                Server
|                                                |
|----- 1. HISTORY_REQUEST ----->| (Εκτέλεση SQL Query)
|    [target="PRIVATE", payload="Aggelina"] | (Φιλτράρισμα βάσει ταυτότητας)
|<----- 2. HISTORY_RESPONSE -----|
|    [payload=[{timestamp, sender, payload}, ...]]
```

Διάγραμμα 3: Ανάκτηση δεδομένων με έλεγχο δικαιωμάτων πρόσβασης.

4. Διαχείριση Σφαλμάτων και Ειδικών Καταστάσεων (Edge Cases)

Το πρωτόκολλο έχει σχεδιαστεί με γνώμονα την παροχή μέγιστης σταθερότητας (fault tolerance) απέναντι σε λανθασμένες ενέργειες των χρηστών ή απρόοπτα σφάλματα δικτύου.

4.1 Αποτροπή Διπλότυπου Username (Collision)

Σε περίπτωση που ένας νέος Client επιχειρήσει να εισέλθει στο σύστημα χρησιμοποιώντας ένα όνομα χρήστη που είναι ήδη δεσμευμένο από ενεργό client ή τη λέξη-κλειδί "SERVER", ο Server απορρίπτει άμεσα το αίτημα. Αποστέλλει ένα πακέτο `CONNECT_RESPONSE` με ένδειξη `status="ERROR"` και `payload="Username already taken"`. Ο Client αναγνωρίζει την αποτυχία, τερματίζει τη διαδικασία σύνδεσης και προτρέπει τον χρήστη να επιλέξει νέο αναγνωριστικό.

4.2 Αποστολή Μηνύματος σε Offline Χρήστη

Όταν ένας χρήστης αποστέλλει ένα ιδιωτικό μήνυμα (`action="PRIVATE"`) με στόχο (`target`) έναν χρήστη που δεν βρίσκεται καταγεγραμμένος στο lookup table του Server, ο Server δεν προωθεί το πακέτο. Ωστόσο, το μήνυμα

αποθηκεύεται κανονικά στη βάση δεδομένων για λόγους ιστορικού, και ο Server απαντά στον αποστολέα με `status="ERROR"` και `payload="User offline"` . Ο Client εμφανίζει κατάλληλο σφάλμα στην οθόνη του (π.χ. `[ERROR]: User offline`).

4.3 Μη Έγκυρες Εντολές (Client-Side Validation)

Για την αποφυγή άσκοπης σπατάλης δικτυακών πόρων και επεξεργαστικής ισχύος, ο Client πραγματοποιεί συντακτικό έλεγχο (parsing) των εντολών πριν από τη δημιουργία του JSON πακέτου. Αν ο χρήστης εισάγει ελλιπή στοιχεία (π.χ. `@all` χωρίς κείμενο ή κενό command), ο Client αποτρέπει την αποστολή datagram και ενημερώνει τοπικά τον χρήστη με μήνυμα σφάλματος ή αναδυόμενο παράθυρο προειδοποίησης.

4.4 Απότομη Αποσύνδεση Client (Network Loss / Crash)

Λόγω της connectionless φύσης του UDP, αν ένας Client τερματιστεί βίαια (π.χ. κρυστάρισμα, απώλεια ρεύματος), δεν αποστέλλεται πακέτο `DISCONNECT` . Για την επίλυση αυτού του προβλήματος, το background thread του Server (`check_timeouts`) εκτελείται ανά 10 δευτερόλεπτα και εξετάζει τις χρονικές σημάνσεις στο λεξικό `last_seen` .

Εάν για οποιονδήποτε χρήστη ισχύει η συνθήκη:

$$T - T > 300 \text{ s current last_seen}$$

τότε ο Server θεωρεί τον Client αποσυνδεδεμένο και προβαίνει αυτόματα στις εξής ενέργειες:

1. Διαγράφει το username από τις ενεργές δομές δεδομένων (`clients` , `user_rooms` , `last_seen`).
2. Εάν ο χρήστης βρισκόταν σε κάποιο chat room, παράγει και αποστέλλει αυτόματα ένα πακέτο `ROOM_ANNOUNCEMENT` προς όλα τα υπόλοιπα μέλη του δωματίου με περιεχόμενο: `"[SERVER]: Ο χρήστης <username> αποσυνδέθηκε λόγω αδράνειας."` .